GRADO EN INGENIERÍA INFORMÁTICA

Indexador

Documentación

Israel Izquierdo Sanchís
DNI:50387916E
Turno: Jueves 9:00 - 11:00

EXPLOTACIÓN DE LA INFORMACIÓN
CURSO 2025/2026

Índice

Introducción 2
Análisis de la solución 3
Algoritmo 3
Estructuras de datos 4
Justificación de la solución elegida 5
Análisis de las mejoras realizadas en la práctica 6
Mejoras algorítmicas y reducción de complejidad 6
Mejoras en la gestión de memoria 6
Mejoras en las operaciones de entrada/salida 7
Análisis de la eficiencia computacional 8
Complejidad Temporal 8
Complejidad Espacial 8
Complejidad Práctica 9

Introducción

En este documento se detalla el diseño, implementación y la optimización algorítmica de la indexación desarrollada en C ++. En el ámbito de la recuperación de la información, tal y como se ha visto en las sesiones de teoría correspondientes a esta parte de la práctica, el indexador representa el núcleo del sistema, siendo este el responsable de una vez el tokenizador nos ha generado esos tokens, lo que hicimos en la práctica anterior, construye un índice invertido que permite relacionar cada término del vocabulario con los documentos en los que aparece, su frecuencia y sus posiciones exactas dentro de esos documentos.

La hipótesis de partida que tenemos para el desarrollo de la práctica del indexador y luego lo que hemos ido desarrollando y optimizando establece que el principal cuello de botella de esta indexación de colecciones masivas de texto no reside únicamente en el coste que hay en evaluar cada palabra, sino que este coste está en la interacción que hay con la RAM y el disco, es decir, entre la memoria principal y el disco y también en las sobrecargas y posibles muros que puede haber también en la memoria. También en esta práctica se asume que el uso de las librerías de C ++ como strings, ifstream o la creación de contenedores dinámicos genera una sobrecarga que si se busca que el código sea lo más rápido posible es inasumible para que esto pase, ya que esta sobrecarga se traduce en una fragmentación extrema en los montículos heap y a su vez provoca fallos continuos en la caché junto con un exceso de llamadas al procesador para pedir la reserva y destrucción.

Bajo esta premisa, el diseño del sistema se ha enfocado en un paradigma orientado a objetos, donde el objetivo de esta práctica ha sido mover la mayor cantidad de trabajo posible a tener estructuras secuenciales y contiguas para así minimizar la creación de variables temporales mejorando así la memoria de la práctica. Por otro lado, se ha buscado también no tener copias por valor y para eso la mayoría se hacen por referencia y con punteros constantes pero esto también es para evitar tener copias innecesarias, y se han aplicado pre reservas de memoria. Con este enfoque, se pretende que este indexador aproveche la jerarquía de memoria y los buffers en el disco para así procesar miles de documentos con muchas apariciones en tiempos relativamente pequeños.

Análisis de la solución

Para solucionar este problema del indexador de una manera estructurada, eficiente y mantenible con el tiempo, el sistema se ha diseñado siguiendo un patrón de una arquitectura dividida en módulos, es decir, con diferentes funciones dentro del código donde se divide este problema en varias capas de abstracción y se respalda con una estructura de datos de muy bajo nivel para así garantizar la eficiencia

Algoritmo

El flujo de la ejecución del indexador se organiza y distribuye a través de la función principal de Indexar e IndexarDirectorio, aunque para explicar el algoritmo solo me hace falta hacerlo con la de Indexar, las demás funciones que tenemos en nuestro código no influyen directamente como tal en la indexación ya que algunas son para impresión, guardado, recuperación, etc, pero la que hace este proceso de indexación de las palabras es la de Indexar. Esta función se divide en las siguientes fases:

En la primera, tenemos una especie de capa de configuración y reserva de caché, donde antes de iniciar la lectura, el sistema lo que hace es estabilizar su estado, y después se pasa con la carga de los stopwords, donde aqui es un ejemplo de una función del código que sirve como apoyo a esta principal para la indexación. Estas palabras de parada se cargan en una estructura de acceso rápido y mediante unas variables bool y numéricas (guardarPos o tipoStemmer) se va mirando. Con esto se evita que el bucle interno que se hace muchísimas veces tenga que tener el trabajo de los punteros a la clase base.

Luego pasamos a leer el archivo que contiene la lista de los documentos a indexar, donde como ya he explicado antes no se usan los métodos de lectura de C ++ ya que son bastante lentos por las comprobaciones internas que se hace, es por ello que se utiliza los punteros nativos de C (FILE*) y un buffer estático de 64 KB. Entonces lo que se hace es que se escanea este bloque que se va leyendo del archivo byte a byte donde se busca los saltos de línea para sacar el nombre de cada fichero de los que hay que leer, donde gestiono manualmente los cortes que puede haber de palabras en los bloques debido a que no entran por tamaño en el buffer, lo que reduce las llamadas al sistema operativo haciendo esto.

Después, una vez se ha sacado a ruta de un documento, se hace una función lambda que lo que va a tener es toda la lógica. Para evitar que la RAM se sature como sí lo haría con un getline, se llama a la función stat para así poder tener el tamaño exacto del fichero en cuanto a número de bytes y con esto lo que hago es ajustar el tamaño del string que reciclo y luego se hace el vuelco a la memoria del archivo solo con un fread.

Tras eso, el bloque pasa por el tokenizador. El flujo se itera secuencialmente sobre la lista de tokens generada y se mira si es una palabra de parada; si existe se incrementa el contador de posición y se ignora mediante un continue, si se pasa y no es una palabra de parada se pasa al algoritmo de stemming.

Por último, se actualiza el índice invertido. Esta es la parte más crítica, donde se utiliza el [] sobre el hash garantizando así que el cálculo solo se haga una vez. Esta parte aprovecha que los documentos se procesan en un orden secuencial para verificar si un documento ya tiene una entrada en la lista de apariciones del término no se recorre la lista entera, lo que se hace es que se comprueba el ID del último elemento insertado y si es un documento nuevo para esa palabra, se inserta al final por lo que la complejidad es O(1) y si ya existía se incrementa la frecuencia y se añade una nueva posición al vector.

Estructuras de datos

Para dar soporte a los algoritmos que se han descrito arriba y evitar que las llamadas a funciones se conviertan en una serie de cuellos de botella, se han seleccionado estructuras de datos que se centran en la reducción de la complejidad temporal y espacial del algoritmo. Las que más destacan en el código son las siguientes:

El índice principal (unordered_map<string, InformacionTermino>) se trata de una tabla hash que garantiza el acceso en una complejidad constante, donde para asegurar esta cota de tiempo lo que hago es que el mapa lo inicializo con una reserva ajustada al tamaño del corpus que tenemos tras varias pruebas y se le aplica un factor de carga máximo, donde lo que hago es que evitar que se tengan colisiones en los términos haciendo que los datos se dispersen.

El filtro de los stopwords (unordered_set<string>) se hace también con una tabla hash donde al no requerir de clave-valor por eso no se pone map sino un set ya que solo se necesita tener validaciones de si existe o no existe y eso al ser binario con un set se hace en un tiempo constante.

La lista de apariciones (vector<pair<int, InfTermDoc>> l_docs) se ha optado por usar un vector en vez de una estructura como las demás ya sea con un mapa ya que el vector garantiza que todos los registros de los documentos para un término estén guardados contiguos en la RAM por lo que esto iba más rápido ya que acelera la recuperación secuencial de estos.

El gestor de posiciones (vector<int> posTerm) guarda las posiciones numéricas de aparición del término. He decidido que sea un vector dinámico ya que funcionaba mejor que como se había propuesto pero sufre también de que la capacidad reservada es en exceso pero asume esa pequeña mejora posible.

Justificación de la solución elegida

El desarrollo de este indexador ha seguido un enfoque iterativo donde se ha ido ajustando el uso de la CPU y de la RAM tras cada modificación que se iba haciendo. Es por ello que a lo largo del proceso se han probado y descartado varias soluciones con el objetivo de tener un equilibrio entre el rendimiento y la memoria.

Primeramente se ha descartado, como ya he mencionado antes, el uso de las funciones de C ++ como ifstream y para leer línea por línea la de getline para procesar el contenido de los documentos. En estas primeras versiones, tenía un código que tras las mediciones llegaba casi a los dos segundos, ya que estas funciones realizaban muchas comprobaciones internas y traducciones que hacian que la CPU tuviera que esperar al disco. Esto se ha sustituido por la función stat para conocer exactamente el tamaño del fichero y con un único fread binario que vuelca todo de golpe en un string reutilizable.

Otra cosa que también se ha descartado también es el cálculo del doble hash en las inserciones. Mi algoritmo lo que hacía antes era que buscaba una palabra con el find y si ya existía en el índice y si no estaba se metia con el emplace. Esto lo descarte porque obligaba al procesador a calcular la huella de cada palabra dos veces, y esto desperdiciaba mucho tiempo para todo el corpus que hay que analizar, esto lo he sustituido por el uso del operador [], que sólo calcula el hash una vez.

También se descartó cambiar la estructura de posTerm por punteros dinámicos o enteros de 16 bits. Se probó usar punteros para ahorrar el overhead de 24 bytes que tienen los vectores vacíos, pero el problema era que al guardar las posiciones se saturaba el heap con miles de llamadas a new, fragmentando la memoria. Al probar con uint16_t se ahorraban unos 1.7 MB de RAM, pero el tiempo de ejecución empeoraba por el trabajo extra que hace un procesador al ser tan pequeñas.

Después se probó pero como todas estas fueron descartadas es el uso de las listas enlazadas o los árboles para guardar la lista de los documentos donde está cada término. Esto pensaba que iba a ser mejor pero esto creaba nodos dispersos en la memoria RAM, provocando constantes fallos en la caché del procesador. Es por eso que apliqué el vector de pares ya que los documentos se miran en orden y siempre la inserción es al final y así puedo mantener toda la memoria contigua y se accede rápido con la instrucción back.

Por último, se descartó también hacer una poda del índice, ya que puse unas variables para analizar todo lo que tenía el corpus, tanto palabras totales analizadas, como documentos, etc. Este análisis me salia que el 20% de las palabras eran raras o tenían errores que solo aparecían una vez en un solo documento, entonces para intentar ahorrar la memoria se probó a borrarlas al final pero esto se descartó porque no pasaba las pruebas de los correctores y si se buscaba una de esas palabras no salia, asi que ni pude conseguir esa mejora de la memoria.

Análisis de las mejoras realizadas en la práctica

Para alcanzar los tiempos de ejecución más bajos posibles y minimizar el consumo de recursos en la memoria RAM, cada iteración y mejora del código se ha ido documentando sistemáticamente en un repositorio git. A lo largo del desarrollo, se han identificado y solucionado diversos cuellos de botella, dividiendo el trabajo empírico en mejoras algorítmicas, gestión de memoria contigua y rediseño de las operaciones de disco.

Mejoras algorítmicas y reducción de complejidad

Primeramente, en las primeras versiones de mi algoritmo, la limpieza de caracteres se realizaba encadenando múltiples llamadas a funciones como replace, esto lo que hacía era que obligaba a la CPU a recorrer la misma cadena varias veces, es por ello que como la mejora a esto era unificar toda esta limpieza en un único bucle for, que esto hacía que pasara muchas veces sobre los mismo a un solo recorrido lineal donde la complejidad se reducía a una lineal.

También, dentro del bucle crítico que había dicho antes en la indexación, acceder continuamente a los atributos de la clase obligaba a resolver el this constantemente entonces para que se evitara este overhead entonces he creado variables y contadores que son locales en el inicio de la función para así que vayan teniendo el estado del objeto y así se acelera el acceso en cada iteración.

En la tokenización, la versión que yo tenía utilizaba las listas lo que añadía mucho tiempo y sobrecarga extra para la indexación, es por ello que como se permite modificar y añadir funciones privadas en el tokenizador, lo que he hecho ha sido desarrollar una función que prescinde de estas listas para mejorar el rendimiento, y para mantener la encapsulación sin que la velocidad se pierda, en el .h del tokenizador se ha puesto que el indexador sea una friend class para que se pueda llamar a esta función privada.

Después, para el stemming y la búsqueda por referencia, lo que se ha hecho es que se ha eliminado la copia innecesaria del string del token cuando no se requiere que la palabra pase por el stemming. Así, la tabla hash se consulta pasando la palabra por referencia con un puntero constante, y si el stemming si es necesario, la modificación de la palabra se aplica directamente in-place, lo que reduce el coste del pre procesamiento.

Mejoras en la gestión de memoria

En este apartado, una de las mejoras es la transición hacia una memoria contigua, esta es una de las mejoras más drásticas. La lista de documentos, el l_docs, pasó de ser un map a un vector de pares de enteros y InfTermDoc. Al garantizar que los documentos se añaden cronológicamente, se eliminaron las búsquedas lineales en favor del método .back(). Del mismo modo, el posTerm se cambió de una lista a un vector, reduciendo drásticamente el overhead de los punteros de los nodos y garantizando que toda la información resida en bloques de memoria contigua.

Por un lado, también he cambiado la parte de la reutilización de la memoria, donde para los casos que no son especiales en el tokenizador se ha utilizado la papelera que se había hecho en la práctica anterior, para así evitar la instanciación de nuevos strings temporales y estabilizando el consumo en el heap.

Por otro lado se ha ido pre reservando la memoria para distintos puntos. Se ha aplicado el reserve tanto para dimensionar la tabla hash principal en base al tamaño del corpus, sino que también al recuperar la indexación desde el disco, pre reservando el tamaño de los vectores antes de leer sus posiciones, esto lo que ha provocado es una mejora bastante grande en la fase de carga del índice.

Mejoras en las operaciones de entrada/salida

Primeramente, otra de las mejores optimizaciones que he hecho ha sido el cambio de la entrada/salida de C ++ por la nativa de C, es decir, se ha descartado el uso de ifstream y se ha sustituido por las funciones nativas de C como fopen o fread, que estas lo que hacen es que tienen menos sobrecarga de sincronización de la librería estándar y no tiene comprobaciones internas por detrás haciendo que el consumo de la memoria sea plano.

También se ha hecho la lectura directa sin utilizar buffers intermedios, es decir, que no se ha usado el memcpy. Esta fue una de las últimas optimizaciones que se han hecho, donde se ha logrado que la lectura de los documentos con un único fread que tira los datos directamente sobre un buffer que está redimensionado, evitando así que se usen copias de memoria redundantes en el bucle y también el uso de buffers de caracteres.

Después, en las primeras versiones, creo que por error del test número 8, tenia que llamar a la función de tokenizar que generaba los archivos temporales .tk, cosa que tanto el tiempo como la memoria fueran a peor, por lo que vi que era un error en el test y eliminé esto tanto para la indexación de los directorios como para la normal. En su lugar, el algoritmo abre el documento original y lo tokeniza directamente volcandolo en la RAM, reduciendo a la mitad el desgaste del disco y las llamadas de E/S

Análisis de la eficiencia computacional

Para validar la eficiencia de la arquitectura implementada, se ha sometido el programa a medición utilizando la herramienta proporcionada./memory). Las pruebas se han realizado procesando el corpus completo de evaluación, con todas las opciones del indexador activadas.

Antes de empezar con el análisis voy a establecer las variables para el problema del indexador de esta práctica:

- N: Número total de palabras a procesar en la colección de documentos.
- V: Tamaño del vocabulario
- D: Número total de documentos en la colección.
- P: Número total de apariciones acumuladas de todos los términos.

Complejidad Temporal

Antes ya se había explicado el funcionamiento de la indexación es por ello que voy a coger esa explicación y voy a ir analizando la complejidad de cada parte, pero ya puedo adelantar que va a tener un comportamiento lineal

En la parte de la lectura del disco, tenemos que la carga del archivo con el fread tiene un coste que es proporcional al tamaño en bytes del documento que hemos leído es por ello que está acotado en una complejidad de O(N).

Luego la parte de normalización y la tokenización correspondiente tenemos que cada carácter se procesa con las tablas pre-calculadas que tenían acceso constante y como ya hemos visto en la práctica anterior y en la teoría de la parte del tokenizador tenía una complejidad también de O(N).

En la actualización del índice, por cada token que se extrae se consulta a tabla hash, donde gracias a la pre reserva con el factor de carga que he explicado anteriormente el número de las colisiones tiende a cero, por lo tanto, el coste de evaluar, insertar o localizar algo de esta hash es de una complejidad constante, O(1).

Y para la parte de la actualización de las apariciones en el vector para la lista de los documentos, como he decidido tener un vector la inserción se realiza siempre al final por lo que el coste va a ser también de O(1).

En conclusión, como resultado del cálculo de la complejidad tenemos un sumatorio que va desde 1 hasta N donde lo que tenemos dentro del sumatorio es un 1 por lo que si lo calculamos tendremos una complejidad de O(N) en el peor de los casos como ya había adelantado antes

Complejidad Espacial

El coste espacial que tiene mi código está determinado por el tamaño de la estructura en la RAM cuando se ha terminado todo el proceso de indexación. Este lo que hace es procesar

un documento, libera el buffer y luego actualiza el índice, por lo que en la memoria final tendremos que hay que almacenar el vocabulario único junto con las listas de documentos que contienen cada palabra y las posiciones, por lo que si calculamos el espacio que se requiere para tener esto nos da una complejidad de O(V+P), y al tener una ausencia de punteros que usaría el map y eso minimiza esta complejidad.

Complejidad Práctica

Para medir la complejidad en la práctica se ha usado la herramienta que se nos ha dejado para ir probando esta práctica, como es el código del memory para medir el tiempo de ejecución y la memoria.

En el tiempo de ejecución, se mide lo que tarda en procesar el corpus de manera completa. Una ejecución que me sale en mi ordenador sería esta:

```
israelizqdo@israelizqdo-HP-250-15-6-inch-G9-Notebook-PC:~/Escritorio/3Carrera/EI/Indexador$ ./memory in
dexador
Ha tardado 0.898859 segundos

Memoria total usada: 66156 Kbytes
  "   de datos: 66020 Kbytes
  "    de pila: 136 Kbytes
```

Este resultado de 0.89 segundos nos confirma el análisis de la complejidad temporal que he explicado antes O(N). Al haber eliminado los bloqueos de lectura del disco y evitar que el hash recalcule posiciones o colisione, el tiempo es lineal respecto al tamaño del corpus, permitiendo procesar miles de documentos por debajo del segundo.

Para el consumo de la memoria, tenemos el resultado que se muestra arriba, lo que demuestra la complejidad espacial analizada antes O(V + P). Esto lo he conseguido con la pre reserva de la capacidad del hash, construir los datos en su sitio definitivo entre otras cosas.